Playwright Intermediate Interview Questions and Answers

1. What are the key advantages of using Playwright over Selenium?

- Multi-Browser Support: Automates Chromium, Firefox, and WebKit using a single API.
- Auto-Waiting Mechanism: Automatically waits for elements to be actionable before performing actions.
- Headless Browser Support: Supports headless execution for faster test runs.
- Network Interception: Allows interception of network requests to mock responses or modify requests.
- Native Mobile Emulation: Supports emulation of mobile devices, enabling responsive testing.

2. Explain the concept of browser contexts in Playwright and their benefits.

Browser contexts allow the creation of multiple independent browser sessions within a single browser instance.

Benefits:

- Isolation: Tests can run independently without affecting each other's state.
- Efficiency: Reduces resource consumption by reusing the same browser instance.
- Parallel Testing: Facilitates parallel execution of tests within the same process.

3. How does Playwright handle different types of waits?

Playwright provides several mechanisms to handle waits:

- Auto-Waiting: Automatically waits for elements to be ready before performing actions.
- Explicit Waits: Methods like page.waitForSelector() wait for specific conditions before proceeding.
- Timeouts: Global and per-action timeouts can be configured to control the maximum wait time.
- Network Idle: page.waitForLoadState('networkidle') waits for the network to be idle before proceeding.

4. Describe how to handle file uploads in Playwright.

To handle file uploads in Playwright:

- Use the setInputFiles method to set the files to be uploaded.

Example:

```
await page.setInputFiles('input[type="file"]', 'path/to/file.txt');
```

- This method sets the value of the file input to the specified file, simulating a user selecting a file for upload.

5. How can you intercept and modify network requests in Playwright?

Playwright allows interception and modification of network requests using the page.route() method:

- Intercept requests and modify responses or block them as needed.

Example:

```
await page.route('**/api/*', route => {
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ key: 'value' }),
  });
});
```

- This is useful for testing how the application handles different server responses.

6. What are selectors in Playwright, and how do they work?

Selectors are used to identify and interact with elements on a webpage.

Playwright supports various types of selectors like CSS, XPath, text, and role selectors.

Example:

```
await page.click('text=Submit');
```

7. Explain the use of fixtures in Playwright.

Fixtures in Playwright are used to set up and tear down resources for tests.

They help in initializing browser instances, contexts, and pages, ensuring consistency across tests.

Playwright Test Runner has built-in fixtures like browser, page, and context.

8. How can you perform mobile device testing in Playwright?

Playwright supports mobile emulation by configuring user agents, screen sizes, and viewport settings.

Example:

```
const iPhone = playwright.devices['iPhone 11'];
const context = await browser.newContext({ ...iPhone });
```

9. How do you capture screenshots in Playwright?

Playwright provides the page.screenshot() method to capture screenshots.

Example:

```
await page.screenshot({ path: 'screenshot.png' });
```

- Screenshots can be full-page or clipped to specific regions.

10. What is the role of trace viewer in Playwright?

Trace viewer in Playwright is a debugging tool that helps visualize test execution.

It provides a timeline of actions, screenshots, and network activity.

Useful for identifying and resolving issues in test scripts.